CHAPTER 18

Nearby Friends

Show which friends are nearby and keep their locations updated in near real time.

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design a **Nearby Friends** feature: opted-in users see a list of friends within a radius (e.g. 5 miles) with approximate distance and a last-updated timestamp. Unlike Proximity Service, **both users move**, so locations change constantly.

Why interviewers ask it: Tests real-time pub/sub, WebSocket, high write QPS for location updates, and ephemeral (TTL) storage.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Which friends count as nearby?	Within 5 miles (configurable).
Distance model?	Straight-line.
Scale?	1 B users, 10% use the feature = 100 M DAU.
Keep location history?	Yes, for ML/analytics.
Inactive friends?	Disappear after 10 minutes of inactivity.

3. Requirements

Functional

- Show nearby friends with distance and timestamp.
- Refresh the list every few seconds.
- Opt-in / opt-out of sharing.

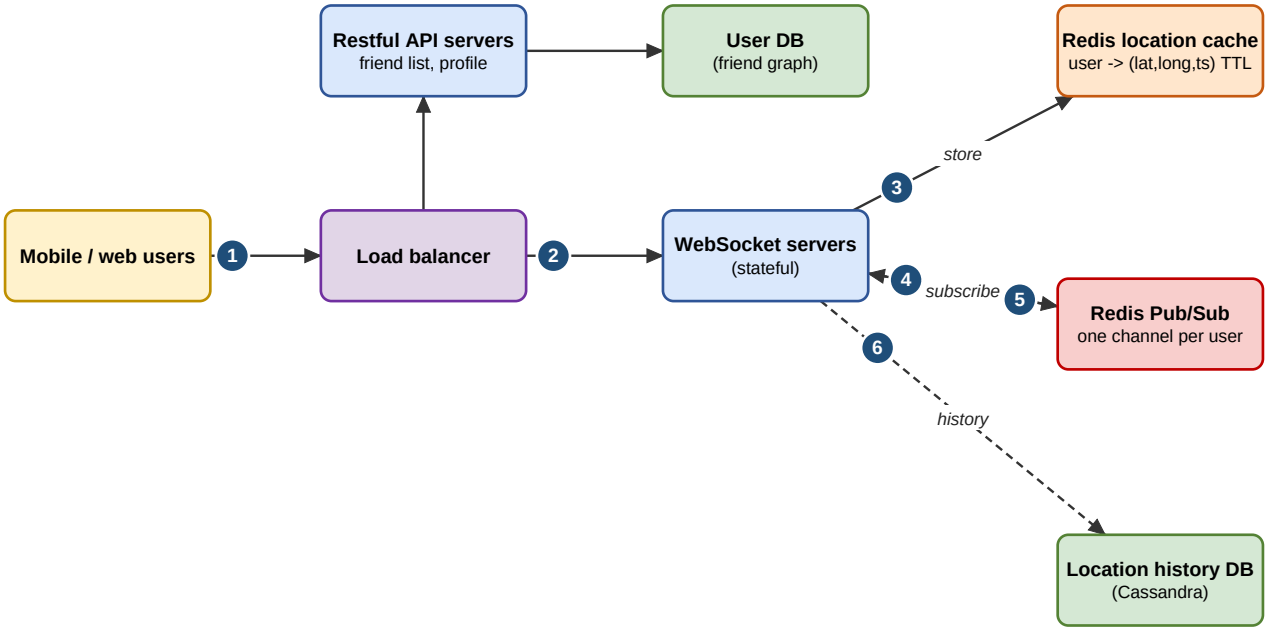
Non-functional

- Low latency for location updates.
- Reliability with tolerated occasional data loss (a lost update is replaced by the next).
- Eventual consistency of locations is fine.

4. Back-of-the-envelope estimation

- 100 M DAU; about 10 M concurrent users. Each reports location every 30 s -> **about 334 K location update QPS** (10 M / 30).
- Assume each user has 400 friends, 10% online and nearby-enabled -> each update fans out to about 40 friends -> **about 14 M location updates/s pushed to clients**.
- Location cache entry about 100 bytes x 10 M active users = 1 GB -> trivial for Redis; history store is the larger volume.

5. High-level design



Step	What happens
1	The app connects through the load balancer. REST calls (friend list, profile) go to the API servers; location updates use a **WebSocket**.
2	The load balancer assigns the user to a **stateful WebSocket server**. On connect, the server loads the friend list and subscribes to each friend's channel.
3	Every ~30 s the client sends its location. The server stores `(lat, long, timestamp)` in the **Redis location cache** with a TTL (10 min), so inactive users disappear on their own.
4	The server **publishes** the update to the user's own channel in Redis Pub/Sub.
5	Every friend's WebSocket server subscribed to that channel receives it, **computes the distance** to its user, and pushes the update down the friend's WebSocket only if within the radius.
6	In parallel the location is appended to the **location history DB** (Cassandra) for analytics and ML. This is off the critical path.

Request flow

- Client sends its location over a **WebSocket** every ~30 s -> WebSocket server stores it in the **Redis location cache** (with a 10-minute TTL) and writes to the history DB.
- The server **publishes** the update to the user's own pub/sub channel.
- Each friend's WebSocket server is **subscribed** to the channels of that user's friends; on receiving an update it computes distance to the subscriber. If within radius, it forwards via WebSocket to the friend's client; otherwise drops.
- On app start: load the friend list, subscribe to each friend's channel, and read their last known locations from the cache to compute the initial list.

6. Deep dive

Why WebSocket and not polling

- Bi-directional and persistent: server must push updates to clients; polling at this scale is wasteful.

Location cache

- Redis `user_id -> {lat, long, timestamp}` with **TTL**; if no update arrives the key expires, so inactive users silently disappear (matches the 10-minute rule). Loss on cache failure is tolerable: next update repopulates.

Pub/Sub channels

- Redis Pub/Sub: lightweight channel creation, nothing persisted, fire-and-forget; each active user has a channel. Memory per channel is small (subscriber list pointers), so 10 M channels is feasible (about 100 GB with many subscribers - shard the pub/sub cluster).
- **Sharding pub/sub:** consistent hashing on `channel_id`; a **service discovery** (ZooKeeper/etcd) holds the hash-ring so WebSocket servers know which Redis node to publish/subscribe to. Resharding is an operational event (stateful, treat like a planned maintenance).

Adding/removing friends and distance calculation

- New friend: callback to the WebSocket server -> subscribe to the friend's channel. The server filters by distance server-side, so clients get only nearby friends.
- Distance: haversine / great-circle on (lat, long); precise enough for 5 miles.

Scaling WebSocket servers

- Stateful; auto-scale in/out carefully: **drain** connections before removing a node. Registry of user -> WebSocket server may be needed.

Nearby random person (extension)

- Use geohash-based channels: users publish to the channel of their geohash cell (and subscribe to the cell + 8 neighbours) to find non-friends.

Privacy and battery

- Opt-in, granular controls, encrypt in transit; reduce update frequency when the user is stationary (adaptive frequency).

7. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Fan-out	Per-user channel (fits friend graph) / geohash channel	Per-user for friends; geohash for strangers.
Storage of current location	Redis (TTL) / DB	Redis for ephemeral, high QPS.
Update interval	Fixed 30 s / adaptive	Adaptive saves battery and load.
Pub/Sub	Redis (fast, no persistence) / Kafka (durable)	Redis is enough; locations are ephemeral.

8. Scaling, reliability, failure modes

- Shard Redis cache by user_id; replicate pub/sub clusters via hash ring.
- Cap friends considered/subscribed to limit fan-out; limit very large friend lists.
- Multi-region deployment: users stay within a regional cluster (cross-region friends via a bridge).

9. Likely follow-up questions

- How do you add and remove Redis pub/sub nodes without dropping subscriptions?
- How would you show nearby non-friends (strangers)?
- How do you handle friends with huge friend lists?
- How do you protect privacy (precision, delays)?

10. Common mistakes

- Re-using the Proximity Service geohash approach without considering movement.
- Persisting every update synchronously in the critical path.
- Ignoring the fan-out multiplier (334 K updates -> 14 M deliveries).

11. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	5 min	"Nearby means within 5 miles, straight-line distance. With 100 M DAU and about 10 M concurrent users reporting every 30 seconds, that is about 334 K location updates per second. With around 40 online friends per user, each update fans out to 40 pushes, so roughly 14 M deliveries per second. A lost update is fine because the next one replaces it."
2. High-level design	10 min	"Clients keep a WebSocket to a stateful server. The server stores the latest location in a Redis cache with a TTL, and publishes it to the user's Redis Pub/Sub channel. Each friend's WebSocket server is subscribed to that channel, calculates distance and pushes to the friend if within range. History goes to Cassandra."
3. Deep dive	25 min	"The tricky part is sharding pub/sub: I use consistent hashing on the channel ID, with the ring stored in a service discovery system. Pub/Sub is chosen because channels are cheap and nothing is persisted. WebSocket servers are stateful, so I drain connections when scaling in. For strangers I'd use geohash-based channels. For privacy I'd make the feature opt-in, and reduce update frequency when the user is stationary."
4. Wrap up	4 min	"Extra points: cap friend counts to limit fan-out, multi-region deployment, and monitoring of fan-out latency. Compared with Proximity Service, the index here is ephemeral and push-based because both ends move."

12. 60-second recap / practice checklist

- WebSocket + Redis location cache (TTL) + Redis Pub/Sub channel per user.
- Publish own update; friends' servers subscribe and filter by distance.
- Shard pub/sub with consistent hashing + service discovery; history in Cassandra.

13. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• Redis Pub/Sub
• Geohash (Wikipedia)
• GeeksforGeeks: Geohashing and quadtrees for location-based services